N

WEEK 2 PROJECT

# Northstar Labs HR Policy Assistant

Course Project Documentation

An end-to-end retrieval-augmented generation application for grounded, page-cited answers over a controlled synthetic HR policy corpus.

**FICTIONAL COMPANY / EDUCATIONAL PROJECT**

Northstar Labs is not a real employer. All policies, company roles, and operating procedures in this project are synthetic and must not be used for real decisions. The application and synthetic policies are not approved for production or employee use.

| 6 | 68 | 15 | 4 |
|---|---|---|---|
| SYNTHETIC POLICIES | INDEXED CHUNKS | FINAL EVALUATION QUESTIONS | TOP EVIDENCE CHUNKS |

| PROJECT OPTION | DOCUMENT DATE | PROJECT TYPE |
|---|---|---|
| Enterprise Policy Q&A Bot | October 18, 2026 | Educational RAG application |

# Contents

Use the linked entries to navigate the report. Section bookmarks are available in compatible PDF viewers.


**1. Project overview** 4

- Selected project option . . . 4
- Project primer . . . 4
- Problem statement . . . 4
- Intended users and interface . . . 4
- Project goals and success criteria . . . 4
- Scope . . . 5

**2. RAG framework** 5

**3. Corpus and data preparation** 6

- Synthetic policy corpus . . . 6
- Why Markdown is converted to PDF . . . 6
- Ingestion and cleaning . . . 6
- Evaluation data . . . 6

**4. System architecture** 7

- End-to-end workflow . . . 7
- LangGraph state and nodes . . . 7
- Dense and lexical retrieval . . . 8
- Reranking and evidence selection . . . 8

**5. Models and implementation stack** 8

- Nebius models . . . 8
- Main technologies . . . 9

**6. Prompting, grounding, and safety controls** 9

- Grounded-answer prompt . . . 9
- Refusal behavior . . . 9

Privacy and error handling . . . 9

**7. Evaluation methodology** **10**

Development calibration . . . 10

Fixed final question set . . . 10

Final measured results . . . 10

Failure analysis . . . 10

Manual review . . . 11

**8. Iterations and design changes** **11**

**9. Development process and AI assistance** **11**

**10. Key learnings** **12**

Retrieval success is not answer success . . . 12

Thresholds require disciplined calibration . . . 12

Valid citations do not prove completeness . . . 12

Automated faithfulness needs manual review . . . 12

Reproducibility is part of system quality . . . 12

**11. Known limitations and future work** **12**

Current limitations . . . 12

Recommended next steps . . . 13

**12. Reproducibility and project evidence** **13**

**13. Conclusion** **14**

**Educational-use disclosure:** Northstar Labs is a fictional company created solely for this course project. It is not a real employer, and the project does not represent or describe the policies of an actual organization. All company names, HR policies, employees, approval roles, and operating procedures in this project are synthetic. The application is a technical demonstration and must not be used for real employment, legal, benefits, tax, medical, or investment decisions.

# 1. Project overview

## Selected project option

This project implements **Project 1: Enterprise Policy Q&A Bot** using a fictional HR and employee policy scenario. The deliverable is a retrieval-augmented generation (RAG) application that answers questions from a controlled policy corpus, supplies page-level citations, and refuses questions that the available documents cannot support.

## Project primer

The Northstar Labs HR Policy Assistant is a Streamlit-based RAG application that answers factual, conditional, and cross-policy questions from six synthetic HR policy PDFs. It returns concise, page-cited answers when evidence is strong enough and a standard referral response when the corpus cannot support an answer.

## Problem statement

Employees often need quick answers to routine policy questions, but a general-purpose language model can invent requirements, omit exceptions, or answer questions that require personal data. The project therefore tests whether a grounded assistant can make policy information easier to navigate without presenting unsupported model output as company policy.

Within the fictional scenario, the application must:

- search only an approved set of policy documents;
- support direct, conditional, and cross-document questions;
- retain important qualifications and exceptions;
- cite the exact policy title and PDF page used;
- refuse unsupported or employee-specific questions safely; and
- expose enough retrieval and evaluation evidence for a reviewer to audit its behavior.

## Intended users and interface

The simulated users are Northstar Labs employees asking questions through an internal Streamlit chat interface. The interface displays the answer or refusal, source citations, retrieved evidence, rerank scores, and processing timings. Conversation history is presentation state only; previous turns are not supplied to the model with a new question.

## Project goals and success criteria

The technical goals are to build a reproducible policy corpus, implement a grounded hybrid RAG pipeline, evaluate it on a fixed question set, and document both successful and unsuccessful behavior. The quality targets are:

| Measure | Target |
|---|---|
| Retrieval recall@4 | At least 85% |
| Faithfulness | At least 95% |

| Measure | Target |
|---|---|
| Refusal accuracy on deliberately unanswerable questions | 100% |
| Citation validity | 100% |
| Citation coverage | 100% |
| End-to-end p95 latency | No more than 8,000 ms |

### Scope

The assistant covers these six synthetic documents:

1. Employee Handbook;
2. Benefits Guide;
3. Leave and Time Off Policy;
4. Remote Work and Information Security Policy;
5. Travel and Expense Policy; and
6. Code of Conduct and Reporting Policy.

The application provides policy navigation, not an individual decision. It does not determine eligibility, approve leave or expenses, investigate complaints, provide accommodations, access employee accounts, calculate tax, or give legal, medical, investment, or benefits advice.

## 2. RAG framework

The following table records the principal design decision at each layer of the RAG system.

| Field | Project decision |
|---|---|
| **Use case** | In the fictional scenario, employees use an internal Streamlit chat interface to ask factual, conditional, and cross-document questions about six HR policies. |
| **Corpus** | Six synthetic English-language policies are maintained as Markdown and published as controlled PDFs. No real employee records or real company policies are included. |
| **Ingestion + cleaning** | A manifest-controlled process generates deterministic PDFs. PyMuPDF extracts text page by page, exact generated headers and footers are removed, and whitespace is normalized without changing policy wording, punctuation, or numbers. |
| **Ingestion + freshness** | The index is rebuilt after a source-policy change. In the simulated operating model, approved changes would be indexed within one business day and reconciled quarterly against a policy register. |
| **Chunking + embedding** | Documents are divided within page and section boundaries into chunks targeting 600 tokens with a 90-token overlap. Nebius `Qwen/Qwen3-Embedding-8B` creates the vectors while headings and page-level citation metadata are preserved. |
| **Retrieve** | Persistent Chroma dense search and BM25 lexical search each retrieve up to eight candidates. Reciprocal-rank fusion combines the result lists without mixing incompatible raw scores. |
| **Rerank** | The best ten fused candidates are scored by normalized cosine similarity between the question and candidate embeddings from `Qwen/Qwen3-Embedding-8B`. The four highest-scoring chunks become answer evidence. |
| **Generate** | Nebius `Qwen/Qwen3-30B-A3B-Instruct-2507` generates an answer only from delimited evidence after the strongest rerank score passes the confidence gate. |
| **Citations and refusal** | Every generated citation must exactly match a retrieved document-title/page pair. Weak evidence, a provider failure, or an invalid citation produces a standard refusal instead of an unverified answer. |

| Field | Project decision |
|---|---|
| **Evaluate** | Five development questions are reserved for threshold calibration. A separate fixed set of 15 questions measures retrieval, faithfulness, refusal behavior, citation quality, latency, and answer completeness through automated and manual review. |

# 3. Corpus and data preparation

## Synthetic policy corpus

The policy corpus was written specifically for this educational project. The editable source files are stored under `data/source/` and registered in `data/source/manifest.json`. The manifest provides stable document identifiers, titles, source paths, and generated PDF names. The six published files are stored under `data/policies/`.

Using synthetic policies provides a controlled environment in which expected answers can be traced to exact source text. It also avoids exposing real employee information or proprietary company policies. References to People Operations, Legal, approvals, service requirements, or employee benefits are parts of the fictional Northstar Labs scenario.

## Why Markdown is converted to PDF

Markdown is the editable source format because it is easy to version, review, and update. The RAG application ingests the generated PDFs because the project requires document and page citations. The deterministic publisher uses fixed page geometry, typography, metadata, and page furniture so that a clean rebuild produces stable page boundaries and repeatable file hashes.

This separation also models a common enterprise workflow: content owners edit a maintainable source format, while employees receive controlled published documents. In this project, both sides of that workflow are simulated.

## Ingestion and cleaning

PyMuPDF extracts one `PolicyPage` object for each nonempty PDF page. Cleaning removes only the exact header and footer generated from manifest data and normalizes whitespace. It does not paraphrase text, alter numbers, or change punctuation. Section-aware chunking then preserves:

- document ID and title;
- section heading;
- PDF page number;
- source path;
- literal policy text; and
- a stable chunk ID derived from provenance rather than model output.

The current corpus produces 68 page-bounded chunks. Page boundaries make citations easier to audit, although they can separate related rules across pages; hybrid retrieval and cross-document testing help address that limitation.

## Evaluation data

Evaluation questions are kept separate from the policy corpus:

- `evaluation/development_questions.json` contains five questions used only for confidence-threshold calibration;
- `evaluation/questions.json` contains the locked 15-question final set; and
- `evaluation/final_manual_reviews.json` contains reviewer decisions tied to the exact final run that produced the reviewed answers.

Keeping development and final questions separate reduces the risk of selecting a threshold that looks good only because it was tuned against the final evaluation set.

# 4. System architecture

RAG PIPELINE AT A GLANCE

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| **INGEST** | **RETRIEVE** | **FUSE** | **RERANK** | **GATE** | **VALIDATE** |
| PDF pages | Dense + BM25 | Rank lists | Nebius cosine | Answer / refuse | Exact citations |

## End-to-end workflow

The application follows this sequence:

1. publish deterministic PDFs from the manifest and Markdown sources;
2. extract and clean text page by page;
3. split text into page- and section-aware chunks;
4. create Nebius embeddings and store them in persistent Chroma;
5. reconstruct a BM25 lexical index from canonical `chunks.json` at startup;
6. retrieve dense and lexical candidates for the current question;
7. fuse duplicate-aware ranked results with reciprocal-rank fusion;
8. rerank the best candidates using embedding cosine similarity;
9. retain the top four excerpts and apply a confidence threshold;
10. either refuse or generate an answer from the selected evidence; and
11. validate every citation before returning the response.

Streamlit and the evaluation script invoke the same compiled graph, so the user interface does not implement a second version of the retrieval or safety logic.

## LangGraph state and nodes

LangGraph coordinates the RAG stages as explicit nodes operating on shared state. The state carries the current question, dense results, lexical results, fused candidates, reranked evidence, decision status, answer text, citations, errors, and timing information.

This structure serves three purposes in the project:

- **Traceability:** each transition from question to evidence to answer can be inspected;
- **Consistency:** the Streamlit interface and evaluator execute the same workflow; and
- **Safety:** the graph can branch to refusal before generation and can replace invalid generated output with a refusal after citation validation.

The main graph stages are question normalization, dense retrieval, lexical retrieval, fusion, reranking, evidence selection, confidence gating, grounded generation, and citation validation.

### Dense and lexical retrieval

Dense retrieval searches Chroma using semantic similarity. It helps when the employee's wording is different from the wording in the policy. BM25 lexical retrieval rewards overlapping terms and is useful for exact phrases, numbers, policy vocabulary, and named concepts.

Each method returns up to eight candidates. Reciprocal-rank fusion uses `1 / (60 + rank)`, merges duplicate chunk IDs, and retains the component rank from each retriever. Using ranks instead of raw scores avoids comparing Chroma similarity values directly with BM25 scores, which have different scales and meanings.

### Reranking and evidence selection

At most ten fused candidates are reranked. The configured Nebius embedding model embeds the question and all candidate texts in one request. Normalized cosine similarity is converted to a bounded 0-1 score, with stable input order used to break score ties. The top four chunks are retained as potential answer evidence.

The strongest rerank score is compared with the calibrated confidence threshold. Below-threshold evidence returns the standard refusal without calling the chat model. This reduces unsupported generation, but the evaluation shows that an overly conservative boundary can also suppress answerable questions.

## 5. Models and implementation stack

### Nebius models

| Purpose | Model | Role in the project |
|---|---|---|
| Embedding | `Qwen/Qwen3-Embedding-8B` | Builds document vectors, embeds questions, and supplies vectors for cosine-similarity reranking. |
| Answer generation | `Qwen/Qwen3-30B-A3B-Instruct-2507` | Produces concise answers from retrieved evidence after confidence gating. |
| Automated faithfulness review | `Qwen/Qwen3-30B-A3B-Instruct-2507` | Judges whether generated claims are supported by the supplied evidence. |

The Nebius gateway centralizes authentication, endpoints, response validation, bounded retries, and sanitized errors. API credentials are loaded from environment variables and are neither committed to the repository nor displayed to users.

### Main technologies

| Technology | Project use |
|---|---|
| Python 3.12 | Application, ingestion, evaluation, and supporting scripts |
| Streamlit | Employee-facing chat demonstration |
| LangGraph | Shared stateful RAG workflow and conditional routing |
| Chroma | Persistent dense-vector index |
| BM25 (`rank-bm25`) | Lexical retrieval |
| PyMuPDF | Page-aware PDF text extraction |
| ReportLab | Deterministic policy PDF generation |
| `httpx` | Nebius API communication |
| Pytest and Ruff | Automated tests, static checks, and formatting verification |

## 6. Prompting, grounding, and safety controls

### Grounded-answer prompt

The generation system message allows the model to use only the evidence supplied by the application. Policy excerpts and the employee question are treated as untrusted data rather than instructions. Evidence blocks include application-generated labels, exact titles, page numbers, section headings, and literal text.

The answer format requires citations such as `[Document Title, p. N]`. The validator accepts only title/page pairs present in the four retrieved evidence chunks. If validation fails, the generated text is discarded and the application returns the standard refusal.

### Refusal behavior

The assistant refuses when:

- no evidence is retrieved;
- the strongest rerank score is below the confidence threshold;
- the question asks for personal or jurisdiction-specific information absent from the corpus;
- the model returns a citation that does not match retrieved evidence; or
- a provider or response-validation error prevents safe completion.

For example, the assistant cannot answer “What is the current balance in my retirement account?” because the synthetic policies contain general benefits information, not individual account data. Retrieving a related benefits page is not sufficient evidence for an employee-specific answer.

### Privacy and error handling

The repository contains no real employee records or personal data. `.env`, authorization values, `data/index/`, and generated evaluation runs are ignored. Errors shown in the interface are sanitized and do not expose provider bodies, request headers, or credentials.

# 7. Evaluation methodology

## Development calibration

Only the five development questions are used to select a confidence threshold. The operational default in configuration is `0.802`, derived from a stable development series. A later provider-backed calibration/final-run pair produced an observed decision boundary near `0.846`. That higher boundary exposed confidence-gate sensitivity and is documented as an evaluation result, not treated as a reason to tune directly on the final set.

## Fixed final question set

The final evaluation contains 15 questions designed to exercise different behaviors:

| Question type | Count | Intended behavior |
|---|---|---|
| Direct | 6 | Retrieve one controlling policy and answer the requested fact with its qualifications. |
| Cross-document | 3 | Combine evidence from two or more policies without merging or losing conditions. |
| Ambiguous | 3 | Identify missing facts and provide a conditional answer when evidence is strong enough. |
| Unanswerable | 3 | Refuse questions requiring personal or jurisdiction-specific information absent from the corpus. |

Each question defines expected source documents and atomic answer points. The evaluator records ranked evidence, rerank scores, answer or refusal outcome, citations, stage timings, an automated faithfulness decision, and a controlled failure category. Manual review then compares the response with the expected points and cited policy pages.

## Final measured results

The provider-backed run dated October 17, 2026 produced these results:

| Measure | Target | Actual | Result |
|---|---|---|---|
| Retrieval recall@4 | >= 85% | 100.0% | Met |
| Faithfulness | >= 95% | 100.0% | Met for generated answers |
| Refusal accuracy | 100% | 100.0% | Met for the three deliberately unanswerable questions |
| Citation validity | 100% | 100.0% | Met |
| Citation coverage | 100% | 100.0% | Met |
| End-to-end p95 latency | <= 8000 ms | 8933.0 ms | Missed by 933.0 ms |

These metrics must be interpreted together. Retrieval recall was perfect on the expected-document criterion, but the confidence gate still refused four answerable questions. Automated faithfulness was perfect only for responses that were generated; it neither credited nor penalized false refusals and did not measure whether every required qualification was included.

## Failure analysis

All four recorded final-set failures occurred after relevant evidence had been retrieved:

| Question | Best relevant evidence | Failure |
|---|---|---|
| Benefit coverage start | Benefits Guide page 1 at `0.845914` | The score fell just below the effective decision boundary, so an answerable direct question was refused. |
| Working from another state | Remote Work and Information Security Policy page 1 at `0.831807` | The gate blocked a conditional answer requesting the employee's location and approval facts. |
| Expensing dinner | Travel and Expense Policy page 2 at `0.834849` | The gate blocked a conditional answer about business purpose, travel status, documentation, and personal expense. |
| Taking tomorrow off | Leave and Time Off Policy page 3 at `0.774530` | The gate blocked clarification and leave-process guidance supported by the retrieved policy. |

These are confidence-gate errors, not missing-document retrieval failures.

### Manual review

Manual evidence review of the same run recorded:

- eight passes;
- three partial answers; and
- four failures.

The partial answers were factually supported but omitted qualifications such as new-hire proration, a lodging exception, or secure information-handling obligations. This demonstrates why citation validity and automated faithfulness do not replace manual completeness review.

Detailed per-question results are available in docs/evaluation-report.md, and the completed run-linked review decisions are stored in `evaluation/final_manual_reviews.json`.

## 8. Iterations and design changes

The project changed in response to implementation findings and evaluation evidence:

1. **Confidence calibration:** development-only calibration replaced the hand-selected `0.35` threshold. Four healthy runs rounded to the `0.802` operational default; a later unhealthy experiment near `0.846` that caused false refusals was not promoted.
2. **Reranker replacement:** a retired public reranker was replaced with Nebius embedding-based cosine similarity, keeping reranking provider-backed and deterministic.
3. **Citation formatting:** grounded-answer prompt version 2 replaced a format that allowed two sources inside one citation bracket. Adjacent single-source citations can be validated exactly.
4. **Instruction precision:** prompt version 3 added explicit protection against reversing obligations and changing modal strength after review found a reversed rule and an overly broad conditional conclusion.
5. **Threshold sensitivity:** the latest run showed that a decision boundary near `0.846` can reject relevant evidence whose score is only slightly lower. The failure was retained and documented rather than hidden through tuning on the final set.

The prompt and implementation decision history is recorded in docs/prompt-log.md.

## 9. Development process and AI assistance

AI-assisted development was used for requirements decomposition, architecture discussion, test review, implementation support, failure analysis, and documentation drafting. The developer retained responsibility for technical decisions and verified the work through:

- offline unit tests;
- Ruff static and formatting checks;
- provider-backed model validation;
- reproducible policy publishing and index construction;
- development-only threshold calibration;
- a locked final evaluation; and
- manual comparison of answers with cited evidence.

The prompt log documents important interactions and resulting changes rather than treating AI output itself as evidence that the system is correct.

# 10. Key learnings

### Retrieval success is not answer success

The final run retrieved the expected policies for all 12 answerable questions, yet the confidence gate refused four of them. A RAG system must therefore evaluate retrieval, gating, generation, and final answer behavior separately.

### Thresholds require disciplined calibration

A threshold should be selected from a healthy development set, not adjusted until final-set scores look favorable. If an answerable development question has no usable score, that issue should be investigated before promoting the resulting threshold.

### Valid citations do not prove completeness

Citation validation proves that a cited location was retrieved. It does not prove that the answer included every exception, deadline, approval requirement, or qualification in the source.

### Automated faithfulness needs manual review

The automated judge measures whether generated claims are supported. It does not reliably identify missing expected points, and it does not score answerable questions that were refused before generation. Manual page-level review remains necessary.

### Reproducibility is part of system quality

Deterministic PDFs, stable chunk identifiers, source hashes, canonical chunk JSON, fixed question sets, versioned prompts, and timestamped evaluation artifacts make failures easier to reproduce and explain.

# 11. Known limitations and future work

### Current limitations

- Northstar Labs and its policies are fictional; the corpus has not been reviewed as real HR or legal guidance.
- The final run produced four false refusals on answerable questions.

- Manual review identified three answers with missing qualifications.
- End-to-end p95 latency exceeded the eight-second target by 933.0 ms.
- The final-run JSON does not persist the exact confidence threshold supplied to the run.
- Public Nebius model availability can change over time.
- The corpus contains only six small English-language policies and does not test access control, multilingual retrieval, large-scale indexing, or real policy-change workflows.
- The automated faithfulness judge is a useful signal, not a substitute for human review.

### Recommended next steps

1. add the effective confidence threshold to evaluation-run metadata;
2. investigate development-set health and recalibrate without using the final set;
3. improve prompt coverage of exceptions and required qualifications;
4. reduce provider and generation latency or revise the performance target with evidence;
5. repeat the locked final evaluation after each model, prompt, chunking, corpus, or threshold change;
6. perform manual review against cited pages for every final-set response; and
7. for any real-world adaptation, replace the synthetic corpus with approved policies and add security, privacy, access-control, content-ownership, and legal-governance reviews.

## 12. Reproducibility and project evidence

The repository separates implementation, source material, evaluation data, and documentation so a reviewer can trace each project claim to supporting evidence.

| Evidence | Location |
|---|---|
| Setup, configuration, commands, and application usage | README.md |
| Streamlit demonstration application | `app.py` |
| RAG implementation | `src/northstar_rag/` |
| Editable synthetic policy sources | `data/source/` |
| Controlled policy PDFs | `data/policies/` |
| Development calibration questions | `evaluation/development_questions.json` |
| Fixed final questions | `evaluation/questions.json` |
| Run-linked manual reviews | `evaluation/final_manual_reviews.json` |
| Prompt and design iteration record | docs/prompt-log.md |
| Final metrics and failure analysis | docs/evaluation-report.md |
| Demonstration walkthrough | docs/video-demo-script.md |
| Model validation script | `scripts/check_models.py` |

| Evidence | Location |
| --- | --- |
| Policy publishing script | `scripts/build_policy_corpus.py` |
| Ingestion script | `scripts/ingest.py` |
| Evaluation script | `scripts/evaluate.py` |

The README contains the exact environment setup, model validation, corpus build, ingestion, evaluation, test, and Streamlit launch commands. Local vector indexes and timestamped evaluation runs are reproducible generated artifacts and are intentionally excluded from version control.

## 13. Conclusion

This course project demonstrates an end-to-end RAG application with a controlled synthetic corpus, hybrid retrieval, provider-backed embeddings and generation, reranking, confidence gating, exact page citations, safe refusal behavior, automated evaluation, and manual evidence review.

The measured results show both strengths and limitations. Retrieval and citation controls performed well, while confidence gating, answer completeness, and latency require further work. Preserving those failures in the project record is important: the purpose of the evaluation is to explain the system's actual behavior, not merely to present favorable metrics.

Northstar Labs remains a fictional setting used to demonstrate the design. Any real deployment would require an approved organization-specific corpus, qualified HR and legal ownership, security and privacy controls, and a fresh evaluation conducted in the intended operating environment.